

Sécurité & OAuth 2.0

Authentification, autorisation et protection des APIs

OAuth 2.0	JWT	HTTPS	CORS
-----------	-----	-------	------

Rate Limiting	Webhooks	Chiffrement
---------------	----------	-------------

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Distinguer authentification et autorisation et leurs mécanismes respectifs
- ✓ Comprendre le protocole OAuth 2.0 Authorization Code Flow en détail
- ✓ Décoder, vérifier et émettre des JSON Web Tokens
- ✓ Sécuriser une API REST : CORS, rate limiting, validation des entrées
- ✓ Vérifier les signatures de webhooks pour prévenir les requêtes frauduleuses
- ✓ Stocker et chiffrer les tokens OAuth de façon sécurisée

Table des matières

Chapitre 1 — Fondamentaux de la sécurité web

- 1.1 Authentification vs autorisation
- 1.2 Les vecteurs d'attaque courants
- 1.3 HTTPS et chiffrement en transit
- 1.4 Principe du moindre privilège

Chapitre 2 — OAuth 2.0 — le protocole en détail

- 2.1 Pourquoi OAuth ?
- 2.2 Les rôles dans OAuth
- 2.3 Authorization Code Flow
- 2.4 Tokens : access, refresh, PKCE

Chapitre 3 — JSON Web Tokens

- 3.1 Structure d'un JWT
- 3.2 Vérification côté serveur
- 3.3 Claims standards et custom
- 3.4 Bonnes pratiques JWT

Chapitre 4 — Sécuriser une API REST

- 4.1 Middleware d'authentification
- 4.2 Validation et sanitisation des inputs
- 4.3 CORS strictement configuré
- 4.4 Rate limiting

Chapitre 5 — Webhooks et stockage des tokens

- 5.1 Vérification de signature HMAC
- 5.2 Chiffrement AES-256-GCM
- 5.3 Variables d'environnement

5.4 Audit logging

Fondamentaux de la sécurité web

1.1 Authentification vs autorisation

Ces deux concepts sont souvent confondus mais sont fondamentalement distincts. L'authentification (authn) répond à la question : 'Qui êtes-vous ?' — c'est la vérification de l'identité. L'autorisation (authz) répond à : 'Avez-vous le droit de faire cela ?' — c'est la vérification des permissions.

Dans Social Seller, Supabase Auth gère l'authentification (vérification du login/mot de passe, émission du JWT). Le Row Level Security de PostgreSQL et le middleware Express gèrent l'autorisation (vérification que l'utilisateur appartient au bon tenant et a les droits nécessaires). Ces deux couches sont indépendantes et complémentaires.

Définition — Zero Trust

Le modèle Zero Trust stipule qu'aucune requête ne doit être considérée comme sûre par défaut, même si elle provient du réseau interne. Chaque requête doit être authentifiée, autorisée, et validée — indépendamment de sa source. Ce modèle remplace l'ancienne approche 'périmètre de confiance' où tout ce qui est dans le réseau interne est considéré sûr.

1.2 Les vecteurs d'attaque courants

Comprendre les attaques courantes est indispensable pour les prévenir. L'OWASP (Open Web Application Security Project) publie chaque année son Top 10 des vulnérabilités web les plus critiques.

Top des vulnérabilités web selon OWASP

Attaque	Description	Prévention
Injection SQL	Code SQL injecté dans un input utilisateur	Requêtes paramétrées, ORMs
XSS	Script malveillant injecté dans la page	Échappement HTML, CSP headers
CSRF	Requête forgée depuis un site tiers	Tokens CSRF, SameSite cookies
IDOR	Accès à des ressources d'un autre utilisateur	Vérification du tenant_id, RLS
Broken Auth	Sessions non expirées, tokens faibles	JWT courts, rotation des tokens
Sensitive Data	Données sensibles en clair dans les logs	Chiffrement, masquage des logs

1.3 HTTPS et chiffrement en transit

HTTPS (HTTP + TLS) chiffre toutes les communications entre le client et le serveur. Sans HTTPS, tout le trafic — tokens, mots de passe, données personnelles — circule en clair et peut être intercepté par n'importe qui sur le réseau (attaque man-in-the-middle). En 2024, HTTPS est obligatoire pour toute application professionnelle. Railway, Supabase et Expo fournissent tous HTTPS automatiquement via Let's Encrypt.

OAuth 2.0 — le protocole en détail

2.1 Pourquoi OAuth ?

OAuth 2.0 résout un problème fondamental : comment permettre à une application tierce d'accéder à des ressources d'un service (Facebook, WhatsApp, Google) au nom d'un utilisateur, sans que l'utilisateur ne communique son mot de passe à l'application tierce ? Avant OAuth, les apps demandaient le login/password de l'utilisateur et s'en servaient directement — ce qui était dangereux et donnait un accès total au compte.

OAuth 2.0 introduit le concept de token d'accès limité : l'utilisateur autorise l'app à accéder à certaines ressources spécifiques (scopes) pour une durée limitée. Si le token est compromis, l'attaquant ne peut accéder qu'aux ressources autorisées, pas au compte entier. L'utilisateur peut révoquer l'accès à tout moment depuis le service original.

2.2 Les rôles dans OAuth

Les 4 rôles OAuth 2.0 dans Social Seller

Rôle OAuth	Dans Social Seller	Rôle concret
Resource Owner	L'utilisateur final	Le marchand qui autorise l'accès
Client	Social Seller	L'app qui demande l'accès
Authorization Server	Meta / Facebook	Émet les tokens après autorisation
Resource Server	Meta Graph API	Répond aux requêtes avec le token

2.3 Authorization Code Flow

L'Authorization Code Flow est le flux OAuth le plus sécurisé pour les applications qui ont un backend. Il se déroule en plusieurs étapes où le code d'autorisation — échangeable contre un token — n'est transmis qu'une seule fois et doit être échangé par le backend (jamais exposé côté client).

Les 5 étapes de l'Authorization Code Flow

```
# Étape 1 : L'app redirige l'utilisateur vers le provider OAuth
# URL construite avec : client_id, redirect_uri, scope, state (anti-CSRF)

GET https://www.facebook.com/v19.0/dialog/oauth
?client_id=1234567890
&redirect_uri=https://backend.railway.app/oauth/facebook/callback
&scope=pages_messaging,pages_read_engagement
```

```
&state=abc123def456 # Token aléatoire stocké en session
&response_type=code

# Étape 2 : L'utilisateur autorise → provider redirige vers redirect_uri
GET https://backend.railway.app/oauth/facebook/callback
?code=AQD...xyz # Code d'autorisation (court-lived, usage unique)
&state=abc123def456 # Doit correspondre au state envoyé

# Étape 3 : Backend échange le code contre les tokens
POST https://graph.facebook.com/v19.0/oauth/access_token
{ code, client_id, client_secret, redirect_uri }
# Réponse : { access_token, token_type, expires_in }

# Étape 4 : Backend stocke le token chiffré en DB
# Étape 5 : Backend redirige l'utilisateur vers l'app mobile
```

■ Erreur fréquente

Ne jamais stocker le `client_secret` côté client (app mobile, navigateur). Il doit rester exclusivement sur le serveur backend. L'exposer permet à n'importe qui d'usurper l'identité de votre application auprès du provider OAuth.

2.4 Tokens : access, refresh, PKCE

L'access token est le token principal, de courte durée (1h-24h), utilisé pour appeler les APIs protégées. Le refresh token est de longue durée (30-90 jours) et permet d'obtenir un nouvel access token sans re-demander l'autorisation à l'utilisateur. PKCE (Proof Key for Code Exchange) est une extension de sécurité pour les apps publiques (mobiles, SPAs) qui ne peuvent pas garder un `client_secret` secret.

JSON Web Tokens

3.1 Structure d'un JWT

Un JWT est une chaîne de trois parties séparées par des points, chacune encodée en Base64URL : header.payload.signature. Le header contient l'algorithme de signature. Le payload contient les claims (informations). La signature garantit l'intégrité : si le payload est modifié, la signature ne correspond plus.

```
# Exemple de JWT décodé

# Header (algorithme de signature)
{ 'alg': 'HS256', 'typ': 'JWT' }

# Payload (claims)
{
  'sub': 'user-uuid-123', # Subject – identifiant de l'utilisateur
  'aud': 'authenticated', # Audience
  'role': 'authenticated', # Rôle Supabase
  'exp': 1735689600, # Expiration (timestamp Unix)
  'iat': 1735686000, # Issued At
}

# Signature = HMAC-SHA256(base64(header) + '.' + base64(payload), secret)
# Si quelqu'un modifie le payload sans connaître le secret,
# la signature ne correspond plus → JWT rejeté

# ■■ Le payload est ENCODÉ (Base64), pas CHIFFRÉ
# N'importe qui peut le décoder – ne jamais mettre de données sensibles
```

3.2 Vérification côté serveur

La vérification d'un JWT en backend doit être rigoureuse. Il ne suffit pas de décoder le payload — il faut vérifier la signature avec le secret, contrôler l'expiration, et vérifier l'audience. Une vérification incomplète est une faille de sécurité critique.

```
// Vérification JWT avec la bibliothèque jsonwebtoken

import jwt from 'jsonwebtoken';
```

```
function verifyToken(token: string): JwtPayload {
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET!, {
      algorithms: ['HS256'], // Spécifier l'algorithme attendu
      audience: 'authenticated',
    });
    return payload as JwtPayload;
  } catch (err) {
    if (err instanceof jwt.TokenExpiredError) {
      throw new UnauthorizedError('Token expired');
    }
    if (err instanceof jwt.JsonWebTokenError) {
      throw new UnauthorizedError('Invalid token');
    }
    throw err;
  }
}
```

■ ■ Attention

Ne jamais utiliser l'algorithme 'none' ni accepter des JWTs auto-signés sans vérification de l'algorithme. Des bibliothèques vulnérables acceptaient des tokens avec alg:'none' qui n'ont pas de signature — permettant à n'importe qui de forger un token valide.

Sécuriser une API REST

4.1 Validation et sanitisation des inputs

Tout input utilisateur doit être considéré comme malveillant jusqu'à preuve du contraire. La validation vérifie que l'input respecte le format attendu (type, longueur, valeurs autorisées). La sanitisation nettoie l'input pour éliminer les caractères potentiellement dangereux. Zod est la bibliothèque de validation TypeScript-first recommandée pour les projets Node.js modernes.

```
import { z } from 'zod';

// Schéma de validation pour créer un produit
const CreateProductSchema = z.object({
  name: z.string().min(1).max(200).trim(),
  price: z.number().int().min(1).max(10_000_000), // en centimes
  stock_quantity: z.number().int().min(0).default(0),
  alert_threshold: z.number().int().min(0).default(5),
  description: z.string().max(1000).trim().optional(),
});

// Middleware de validation réutilisable
function validate<T>(schema: z.ZodSchema<T>) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return res.status(400).json({
        error: 'Validation failed',
        details: result.error.flatten(),
      });
    }
    req.body = result.data; // Body validé et typé
    next();
  };
}

// Application sur la route
router.post('/', validate(CreateProductSchema), async (req, res) => {
```

```
// req.body est garanti valide ici  
});
```

4.2 CORS strictement configuré

CORS (Cross-Origin Resource Sharing) contrôle quels domaines peuvent appeler votre API depuis un navigateur. Une configuration CORS trop permissive (origin: '*') avec authentification est une faille de sécurité. Limitez toujours l'origin aux domaines légitimes.

```
import cors from 'cors';  
  
const allowedOrigins = [  
  'http://localhost:8081', // Expo web dev  
  'https://socialseller.djiguitech.com', // Production  
];  
  
app.use(cors({  
  origin: (origin, callback) => {  
    // Autoriser les requêtes sans origin (mobile, Postman, tests)  
    if (!origin || allowedOrigins.includes(origin)) {  
      callback(null, true);  
    } else {  
      callback(new Error(`CORS: origin ${origin} not allowed`));  
    }  
  },  
  credentials: true, // Nécessaire pour les cookies  
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],  
}));
```

Webhooks et stockage des tokens

5.1 Vérification de signature HMAC

Les webhooks sont des appels HTTP envoyés automatiquement par un service externe quand un événement survient (nouveau message WhatsApp, paiement reçu, etc.). Comme tout le monde peut envoyer une requête HTTP à votre endpoint, il faut vérifier que la requête vient bien du bon service. Le mécanisme standard est la signature HMAC : le service signe le body de la requête avec un secret partagé, et votre backend vérifie la signature.

Vérification HMAC avec protection contre les timing attacks

```
import crypto from 'crypto';

// Vérification de signature webhook Meta (WhatsApp/Facebook)
function verifyWebhookSignature(req: Request, res: Response, next: NextFunction) {
  const signature = req.headers['x-hub-signature-256'] as string;
  if (!signature) {
    return res.status(401).json({ error: 'Missing signature' });
  }

  const [algo, hash] = signature.split('=');
  if (algo !== 'sha256') {
    return res.status(401).json({ error: 'Unsupported algorithm' });
  }

  const expected = crypto
    .createHmac('sha256', process.env.META_APP_SECRET!)
    .update(req.rawBody) // Body brut, avant parsing JSON
    .digest('hex');

  // timingSafeEqual évite les timing attacks
  const matches = crypto.timingSafeEqual(
    Buffer.from(hash),
    Buffer.from(expected)
  );

  if (!matches) {
    return res.status(401).json({ error: 'Invalid signature' });
  }
}
```

```
}  
  
next();  
  
}
```

5.2 Chiffrement AES-256-GCM

Les tokens OAuth (access_token, refresh_token) récupérés lors du flux OAuth sont des credentials sensibles. Les stocker en clair en base de données est une erreur de sécurité grave — si la DB est compromise, tous les tokens sont exposés. La solution est de les chiffrer avant stockage avec AES-256-GCM, un algorithme de chiffrement symétrique authentifié.

Chiffrement symétrique AES-256-GCM pour les tokens OAuth

```
import crypto from 'crypto';  
  
const ALGORITHM = 'aes-256-gcm';  
const KEY = Buffer.from(process.env.TOKEN_ENCRYPTION_KEY!, 'base64'); // 32 bytes  
  
export function encryptToken(plaintext: string): string {  
  const iv = crypto.randomBytes(12); // 96-bit IV – différent à chaque chiffrement  
  const cipher = crypto.createCipheriv(ALGORITHM, KEY, iv);  
  
  const encrypted = Buffer.concat([  
    cipher.update(plaintext, 'utf8'),  
    cipher.final(),  
  ]);  
  
  const authTag = cipher.getAuthTag(); // Tag d'authenticité (16 bytes)  
  
  // Stocker : iv + authTag + ciphertext en base64  
  return Buffer.concat([iv, authTag, encrypted]).toString('base64');  
}  
  
export function decryptToken(ciphertext: string): string {  
  const buf = Buffer.from(ciphertext, 'base64');  
  const iv = buf.subarray(0, 12);  
  const authTag = buf.subarray(12, 28);  
  const data = buf.subarray(28);  
  
  const decipher = crypto.createDecipheriv(ALGORITHM, KEY, iv);  
  decipher.setAuthTag(authTag); // Vérification d'intégrité  
  
  return decipher.update(data) + decipher.final('utf8');  
}
```

■ À retenir

- ✓ Authentification = qui êtes-vous ? | Autorisation = avez-vous le droit ?
- ✓ OAuth 2.0 : l'utilisateur autorise l'app à accéder à ses ressources sans partager son mot de passe
- ✓ JWT : header.payload.signature — le payload est encodé, pas chiffré
- ✓ Toujours vérifier la signature JWT côté serveur — ne jamais faire confiance au payload seul
- ✓ Valider tous les inputs avec Zod avant de les utiliser
- ✓ Vérifier les signatures HMAC des webhooks avec timingSafeEqual (anti timing attack)

— ■ Exercices

Exercice 1 — Implémenter OAuth complet

Construire un flux OAuth 2.0 Authorization Code Flow de bout en bout.

- Générer et stocker un state anti-CSRF avant la redirection
- Construire l'URL d'autorisation avec client_id, scope, state, redirect_uri
- Implémenter le callback : vérifier le state, échanger le code contre un token
- Stocker le token chiffré en DB avec AES-256-GCM

Exercice 2 — Middleware de sécurité

Construire une couche de sécurité complète pour une API Express.

- Middleware d'auth JWT avec vérification complète (signature, expiration, audience)
- Validation Zod sur toutes les routes POST/PATCH
- CORS avec whitelist d'origins
- Rate limiter : 100 req/min par tenant

Exercice 3 — Webhook sécurisé

Implémenter un endpoint de webhook avec vérification de signature.

- Configurer express.json() avec verify callback pour conserver le rawBody
- Vérifier la signature HMAC avec timingSafeEqual
- Retourner 200 immédiatement, traiter le webhook de façon asynchrone
- Logger les webhooks invalides avec l'IP source

Bibliographie & Ressources

Spécifications

- oauth.net/2 — spécification OAuth 2.0 (RFC 6749)
- jwt.io — décodeur JWT en ligne + documentation
- owasp.org/www-project-top-ten — OWASP Top 10

Livres

- Justin Richer & Antonio Sanso — OAuth 2 in Action (Manning, 2017)
- Okta — The Definitive Guide to OAuth 2.0 (gratuit en ligne)

Outils

- jwt.io — vérifier et décoder des JWTs
- requestbin.com — inspecter les webhooks entrants
- zod.dev — documentation Zod